UNIVERSIDAD
Popular del cesar

Ingeniería de Sistemas



Patrón estructural: FlyWeight

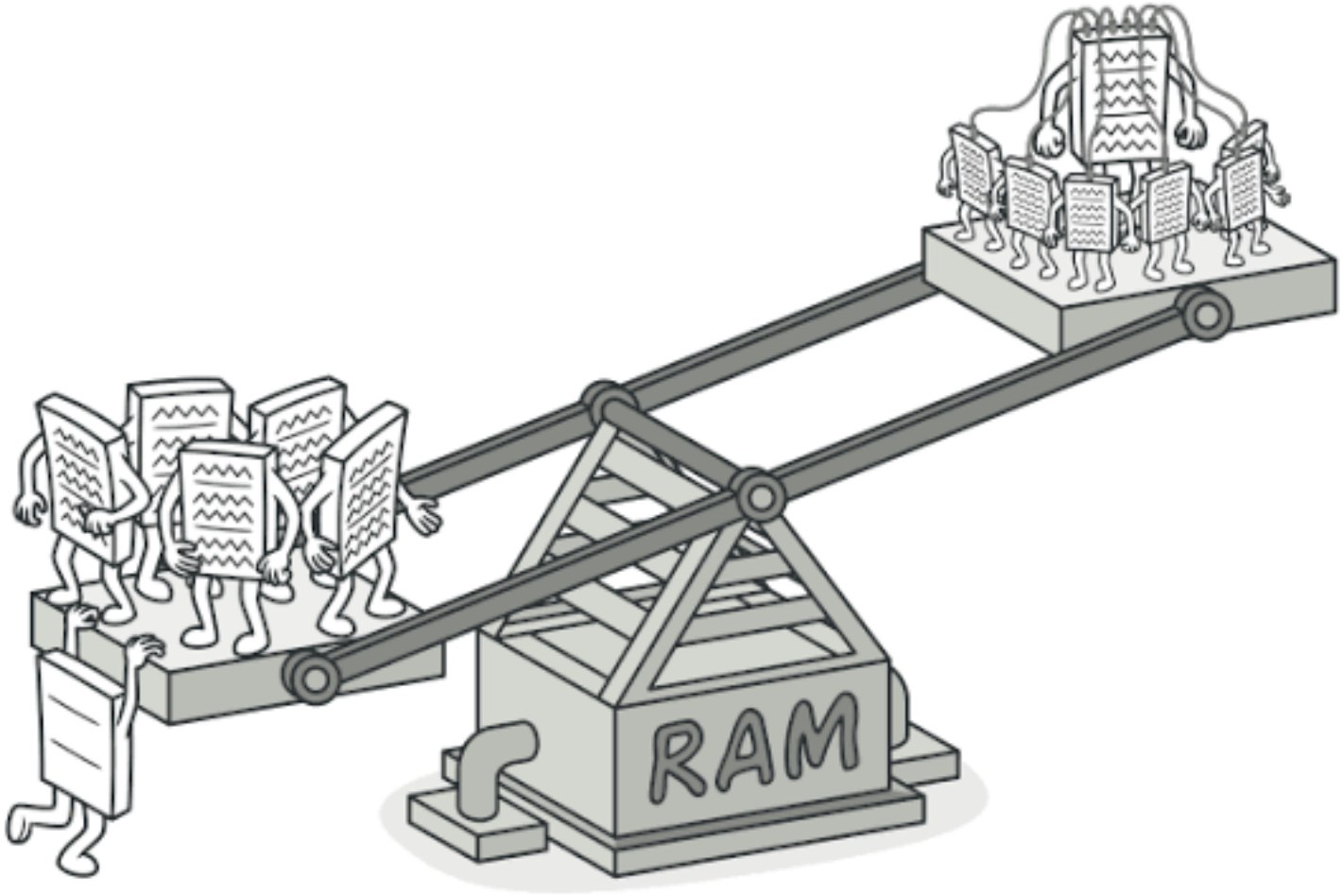
Fabio Andrés Paredes Pineda
Harlevis Pacheco Torres
Cristian Santana
Eder David Martínez Molina

Profesor: Jairo Seoanes · Universidad Popular del Cesar · 2026

FlyWeight

1. Propósito
2. Casos de uso
3. Aplicabilidad
4. Estructura (Diagrama de clases)
5. Código (Antes y Después)

PROPOSITO



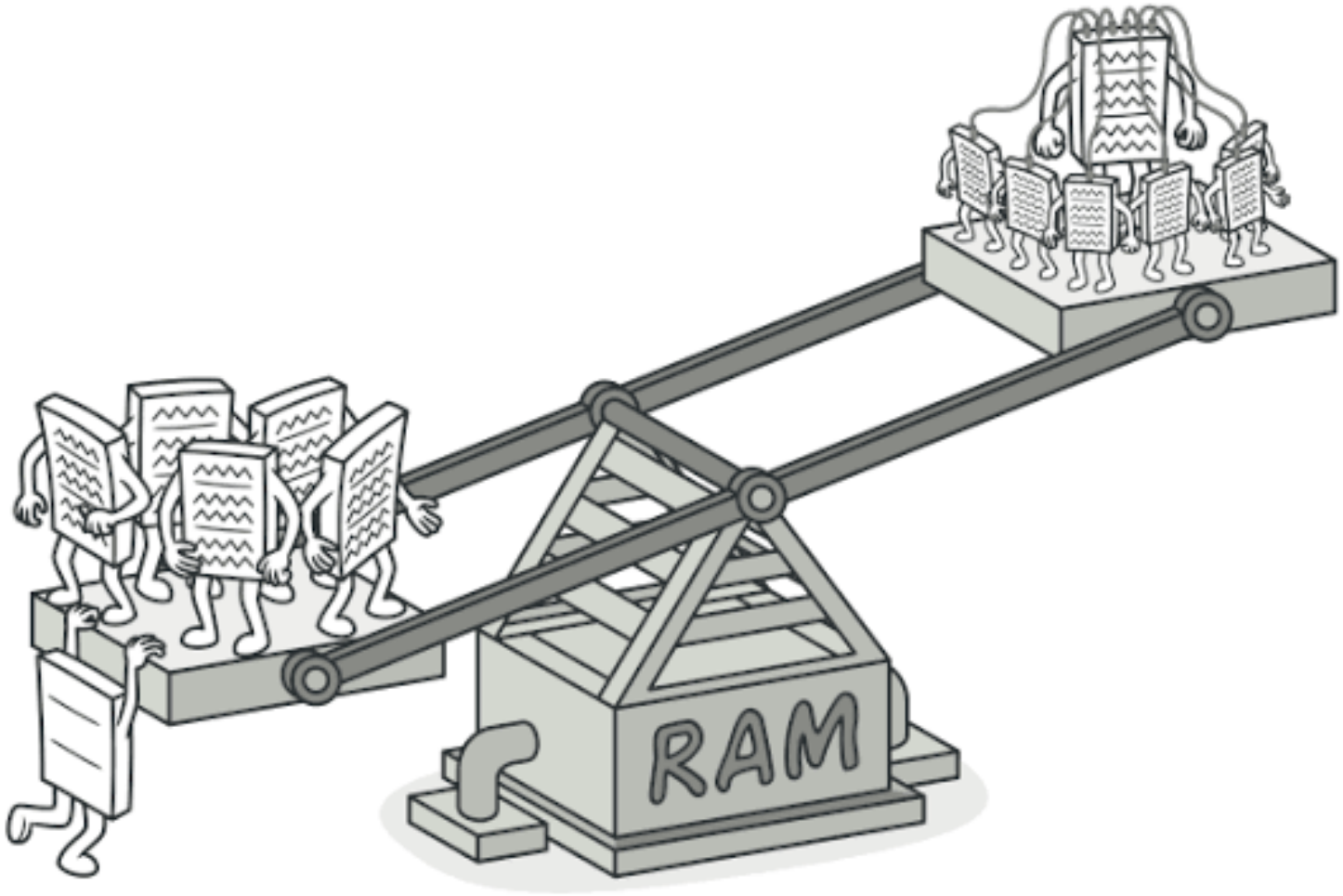
Beneficio Clave	¿Cómo lo logra?
Reducción de consumo	Reutiliza datos idénticos en lugar de duplicar información en cada objeto.
Soporte masivo	Permite al sistema renderizar o procesar millones de entidades simultáneamente.
División de datos	Separa el objeto en un estado Intrínseco (Compartido) y Extrínseco (Contextual).

Flyweight es un **patrón de diseño estructural** que permite mantener más objetos dentro de la cantidad disponible de RAM compartiendo las partes comunes del estado entre varios objetos en lugar de mantener toda la información en cada objeto.

Casos de usos



APLICABILIDAD



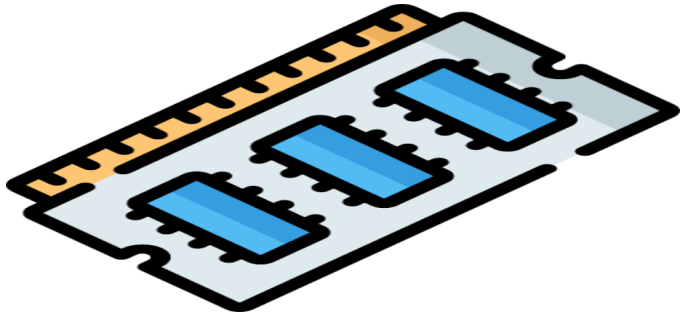
¿Cuándo aplicar flyweight?

UNIVERSIDAD Popular del cesar

Solo cuando la memoria es el cuello de botella



Los objetos tienen estado duplicado que se puede extraer y compartir.



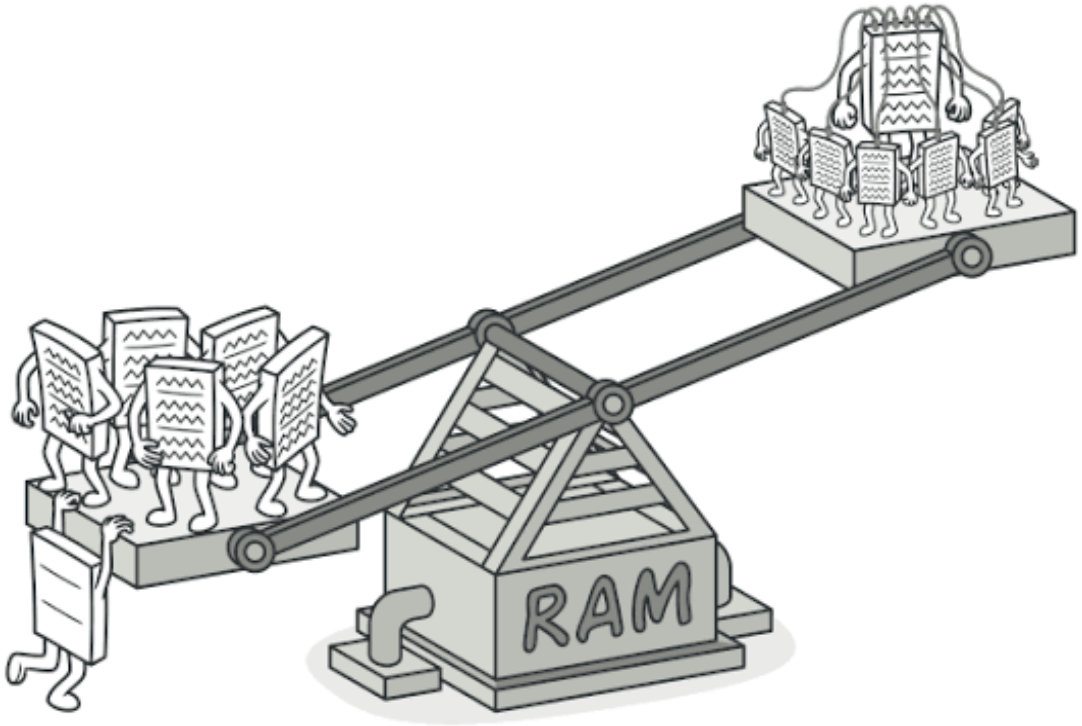
Los objetos consumen toda la RAM del dispositivo objetivo.



La aplicación necesita crear una cantidad enorme de objetos similares.

La idea clave para decidir

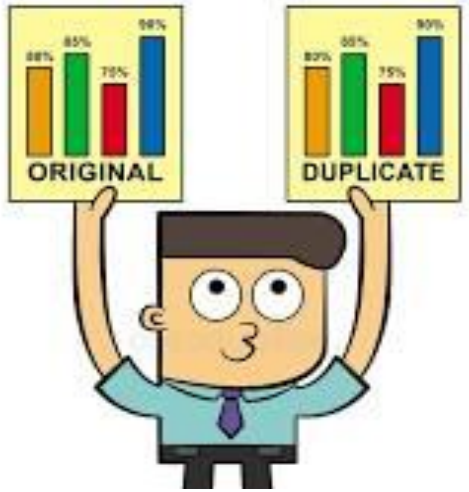
Estado intrínsico	Estado extrínsico
Se repite muchos objetos	Es único de cada objeto
No cambia	Cambia con el tiempo
Vive dentro del flyweight	Lo guarda el cliente o contexto y se pasa como parámetro
Se comparte	No se comparte
Ejemplo: color,sprite,textura	Ejemplo: coordenadas, velocidad, dirección



Aplicaciones reales

UNIVERSIDAD Popular del cesar

Color y sprite de balas, enemigos, personajes





Lista de reproducción

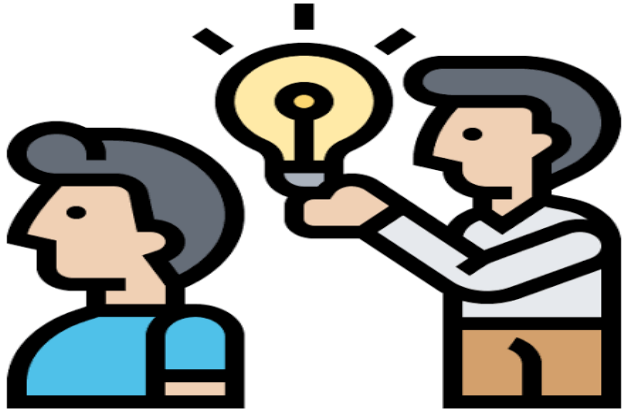
Estilo de caracteres como fuente, tamaño, color



Cuando no aplicarlo



Objetos cuyos datos son casi todos distintos

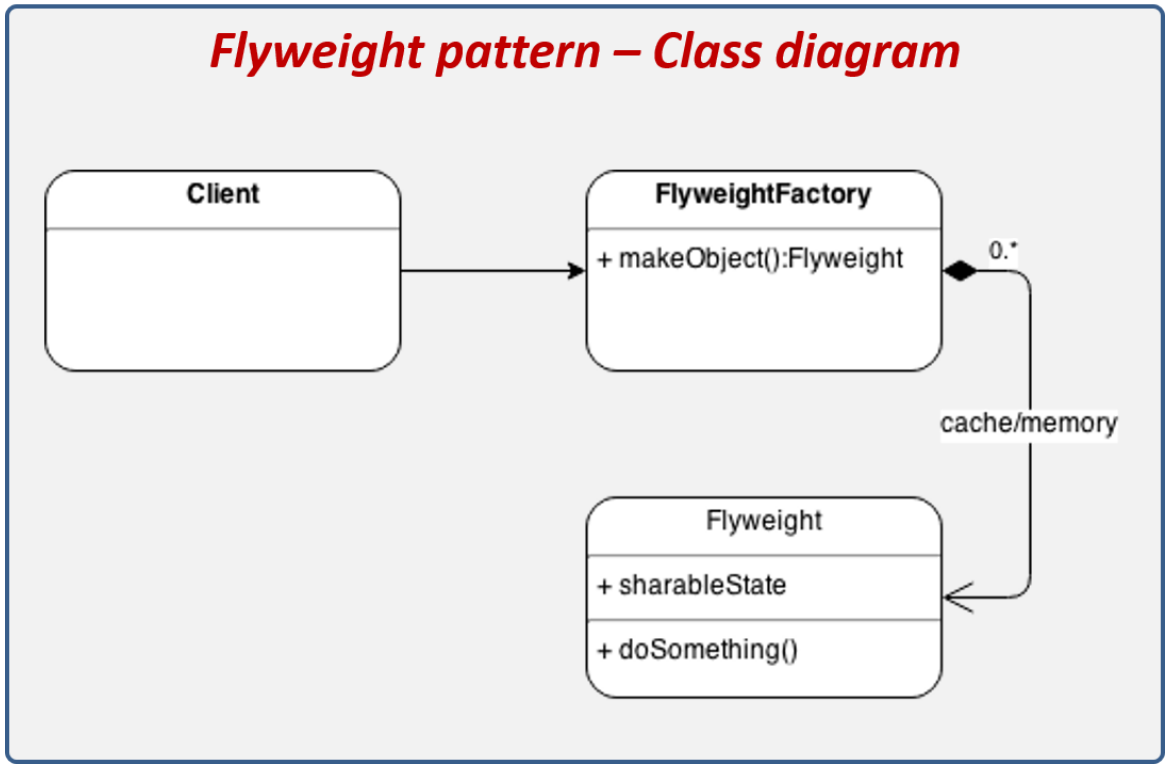


Objetos que necesitan cambiar su estado compartido



No hay un problema real de memoria

UML FlyWeight



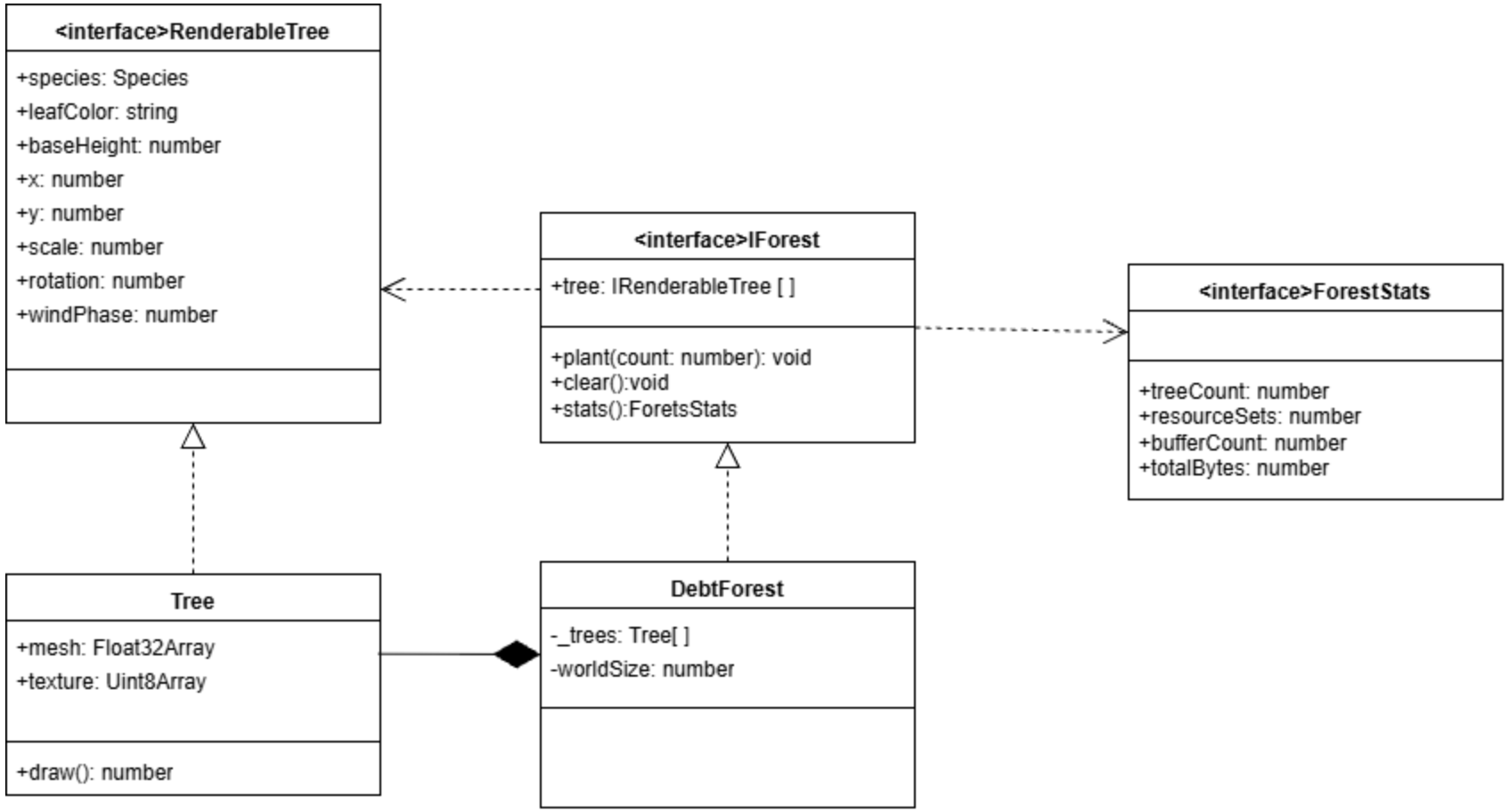
Client: Objeto que dispara la ejecución.

FlyweightFactory: Fábrica que utilizaremos para crear los objetos Flyweight u objetos ligeros.

Flyweight: Corresponde a los objetos que deseamos reutilizar con el fin de que nuestros objetos sean más ligeros.

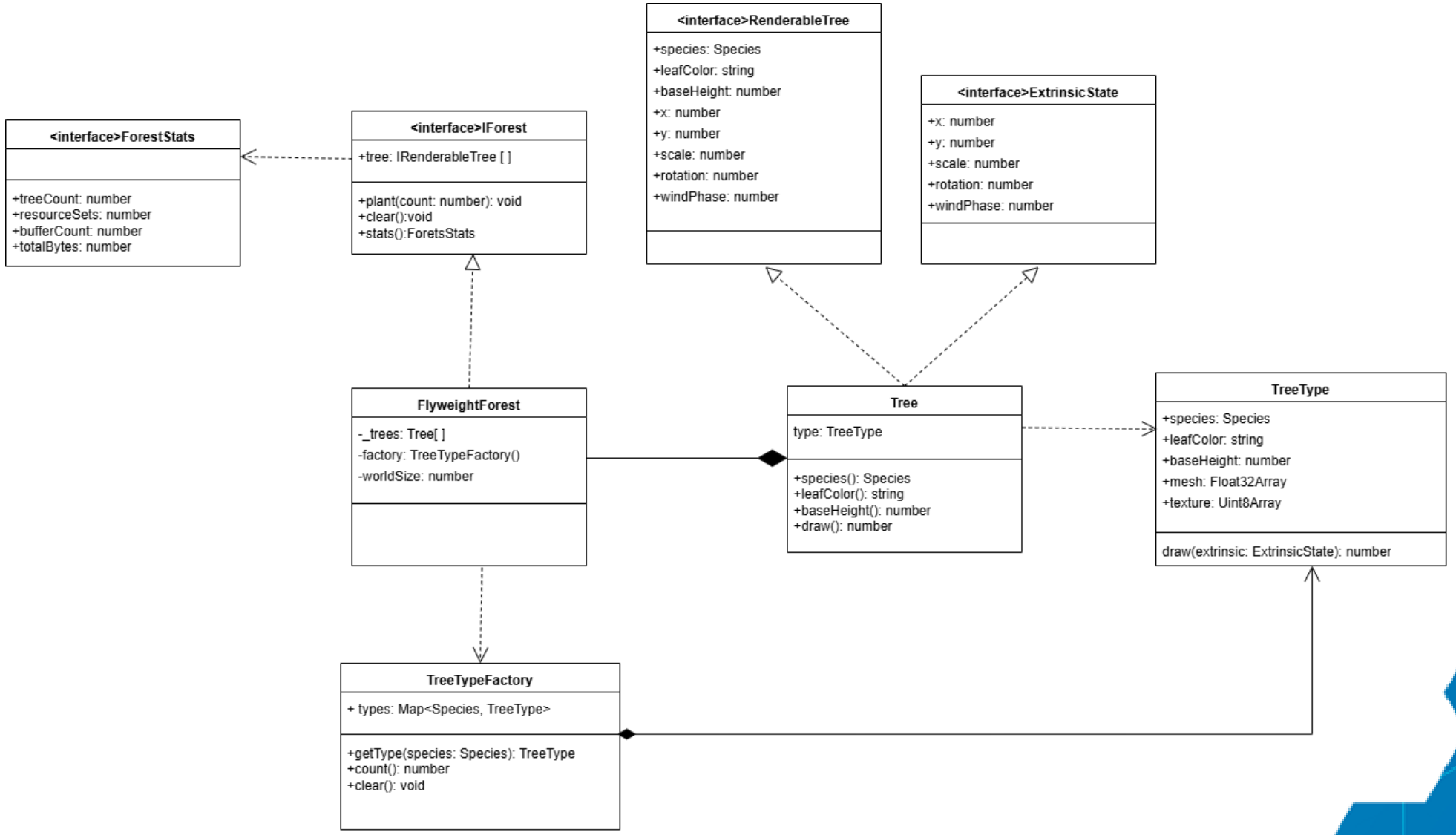
Estructura diagrama de clases
Antes

UNIVERSIDAD Popular del cesar



Estructura diagrama de clases Despues

UNIVERSIDAD Popular del cesar



Código Antes

UNIVERSIDAD Popular del cesar

```ts
export class Tree implements IRenderableTree {
  // --- Estado intrínseco (repetido en cada árbol) ---
  public readonly species: Species;
  public readonly leafColor: string;
  public readonly baseHeight: number;
  public readonly mesh: Float32Array; // ~60 KB duplicados
  public readonly texture: Uint8Array; // ~64 KB duplicados

  // --- Estado extrínseco (único por árbol) ---
  public x: number;
  public y: number;
  public scale: number;
  public rotation: number; // radianes
  public windPhase: number; // radianes, desfasa el balanceo de cada árbol
```

```ts
export class DebtForest implements IForest {
  private _trees: Tree[] = [];

  /** @param worldSize lado del mundo cuadrado (en unidades de mundo). */
  constructor(private readonly worldSize: number = 1000) {}

  public get trees(): readonly IRenderableTree[] {
    return this._trees;
  }
```

```ts
export interface IRenderableTree {
  readonly species: Species;
  readonly leafColor: string;
  readonly baseHeight: number;
  readonly x: number;
  readonly y: number;
  readonly scale: number;
  readonly rotation: number;
  readonly windPhase: number;
}
```

```ts
export interface IForest {
  readonly trees: readonly IRenderableTree[];
  plant(count: number): void;
  clear(): void;
  stats(): ForestStats;
}
```

Código Después

UNIVERSIDAD Popular del cesar

```
export class Tree implements IRenderableTree, ExtrinsicState {
  // Referencia compartida (8 bytes), no una copia de los buffers
  private readonly type: TreeType;

  // --- Estado extrínseco (único por árbol) ---
  public x: number;
  public y: number;
  public scale: number;
  public rotation: number;
  public windPhase: number;
```

```
export class TreeTypeFactory {
  private readonly types = new Map<Species, TreeType>();

  public getType(species: Species): TreeType {
    let type = this.types.get(species);
    if (!type) {
      type = new TreeType(species);
      this.types.set(species, type);
    }
    return type;
  }

  /** Cantidad de flyweights distintos en memoria. */
  public count(): number {
    return this.types.size;
  }

  /** Vacía la caché para que el GC libere los buffers. */
  public clear(): void {
    this.types.clear();
  }
```

```
export class TreeType {
  public readonly species: Species;
  public readonly leafColor: string;
  public readonly baseHeight: number;
  public readonly mesh: Float32Array; /
  public readonly texture: Uint8Array;
```

```
// --- Estado intrínseco: se delega en el flyweight ---
public get species(): Species {
  return this.type.species;
}

public get leafColor(): string {
  return this.type.leafColor;
}

public get baseHeight(): number {
  return this.type.baseHeight;
}

public draw(): number {
  return this.type.draw(this);
}
```

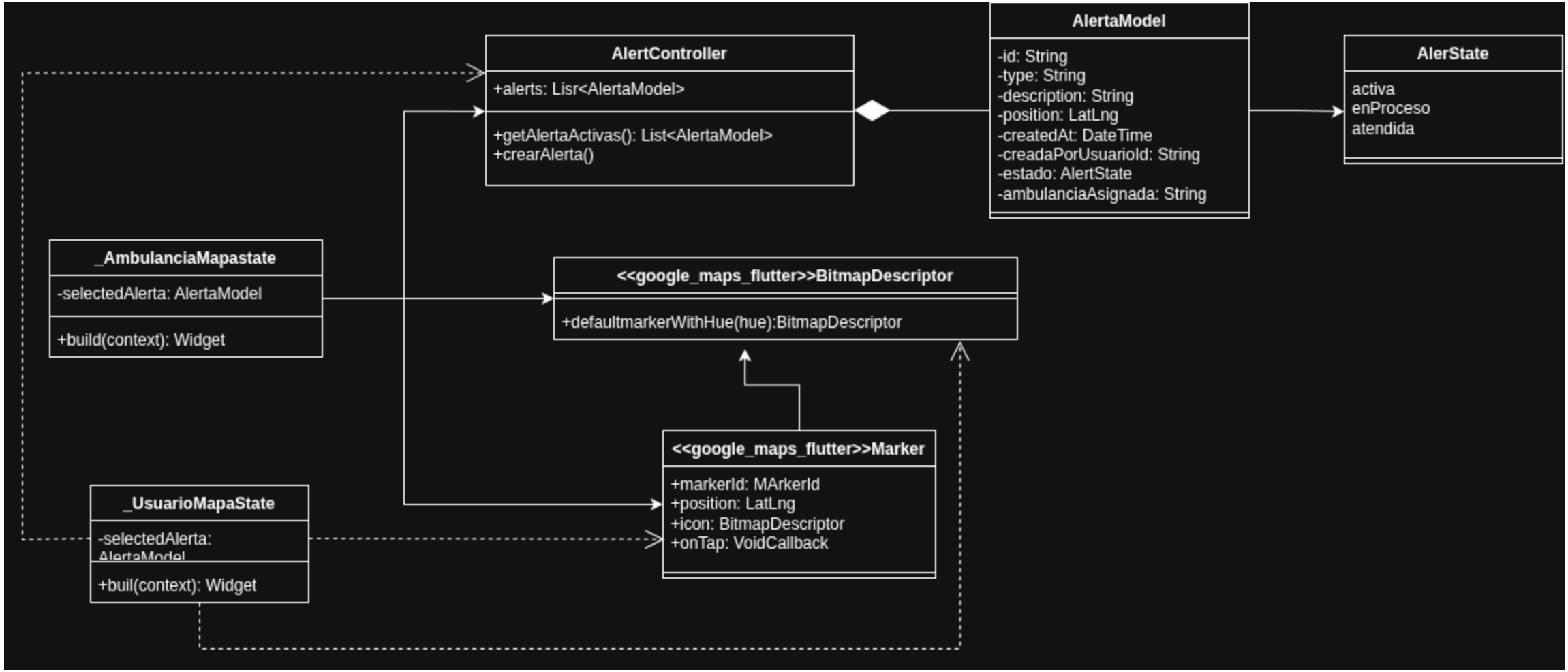
```
export class FlyweightForest implements IForest {
  private _trees: Tree[] = [];
  private readonly factory = new TreeTypeFactory();

  /** @param worldSize lado del mundo cuadrado (en unidades de mundo). */
  constructor(private readonly worldSize: number = 1000) {}

  public get trees(): readonly IRenderableTree[] {
    return this._trees;
  }
```

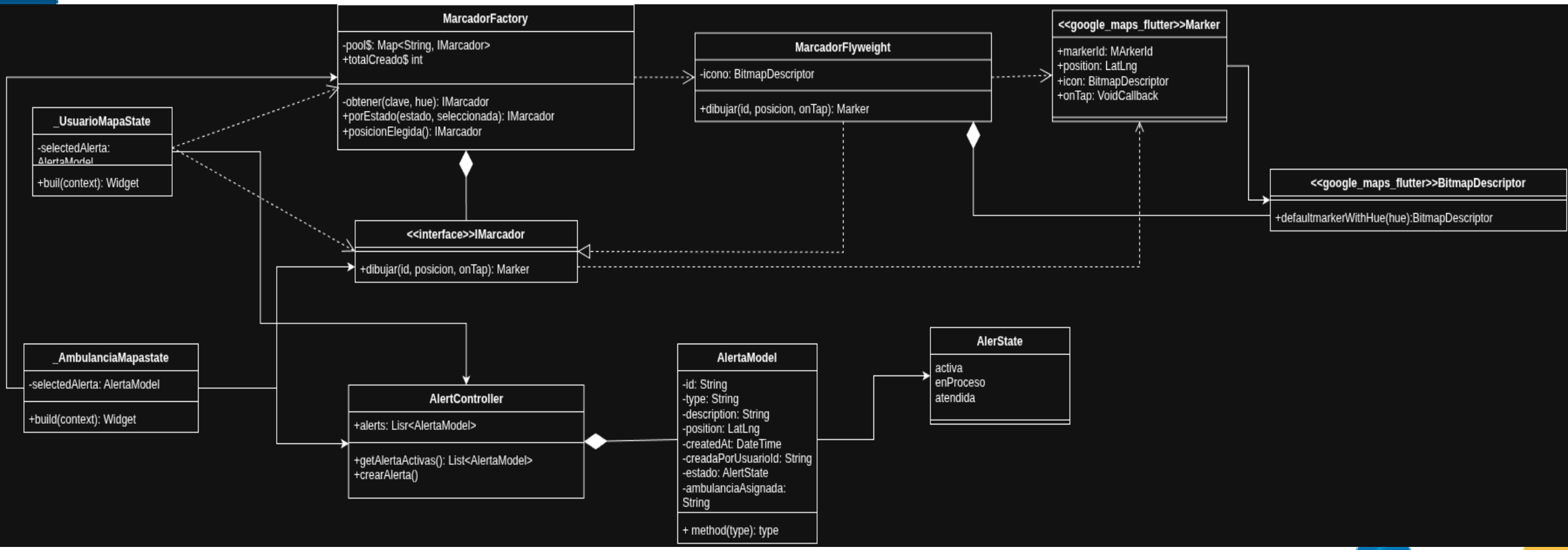
Diagrama de clases proyecto

ESTRUCTURA



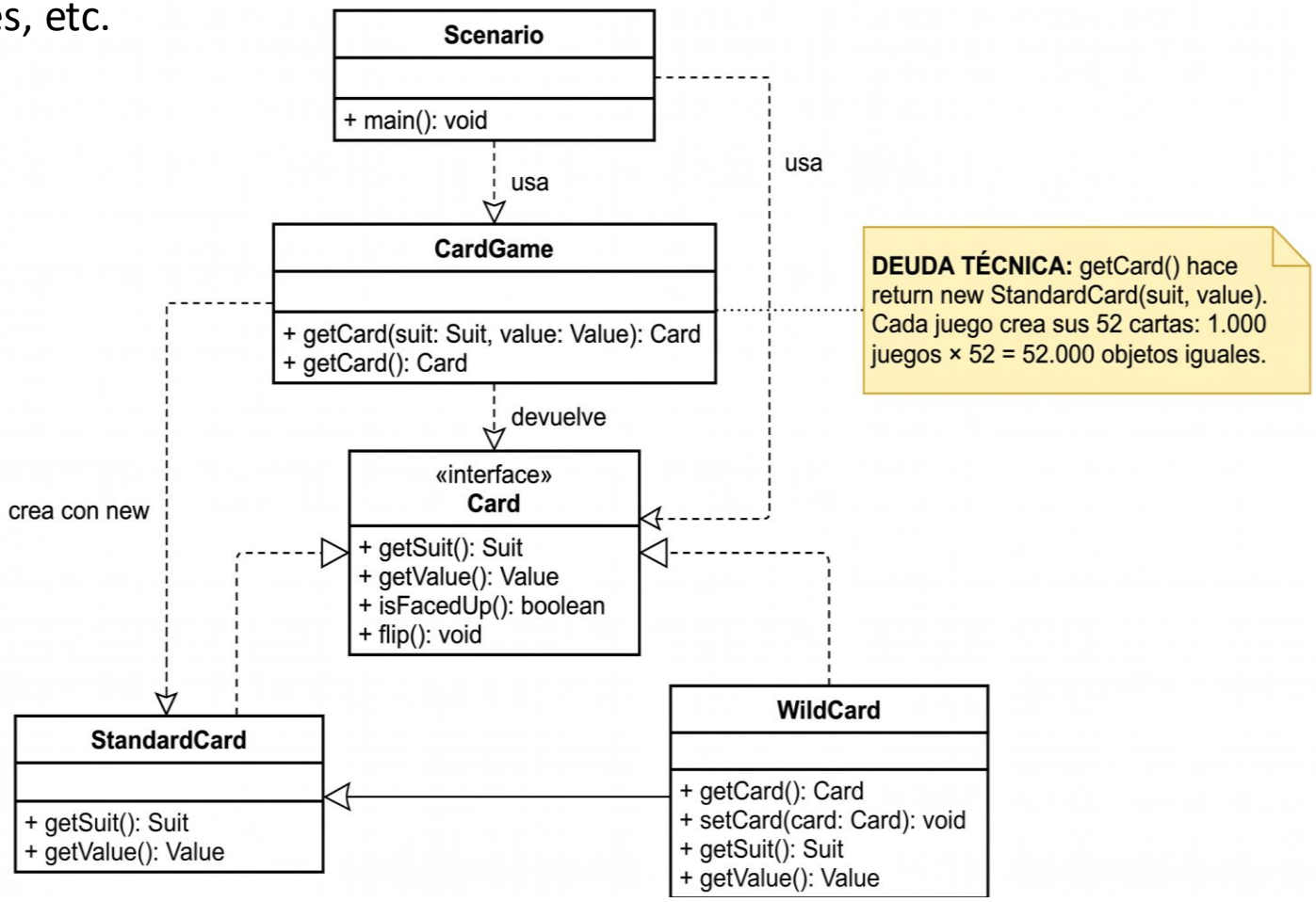
Estructura diagrama de clases con Flyweight

UNIVERSIDAD Popular del cesar



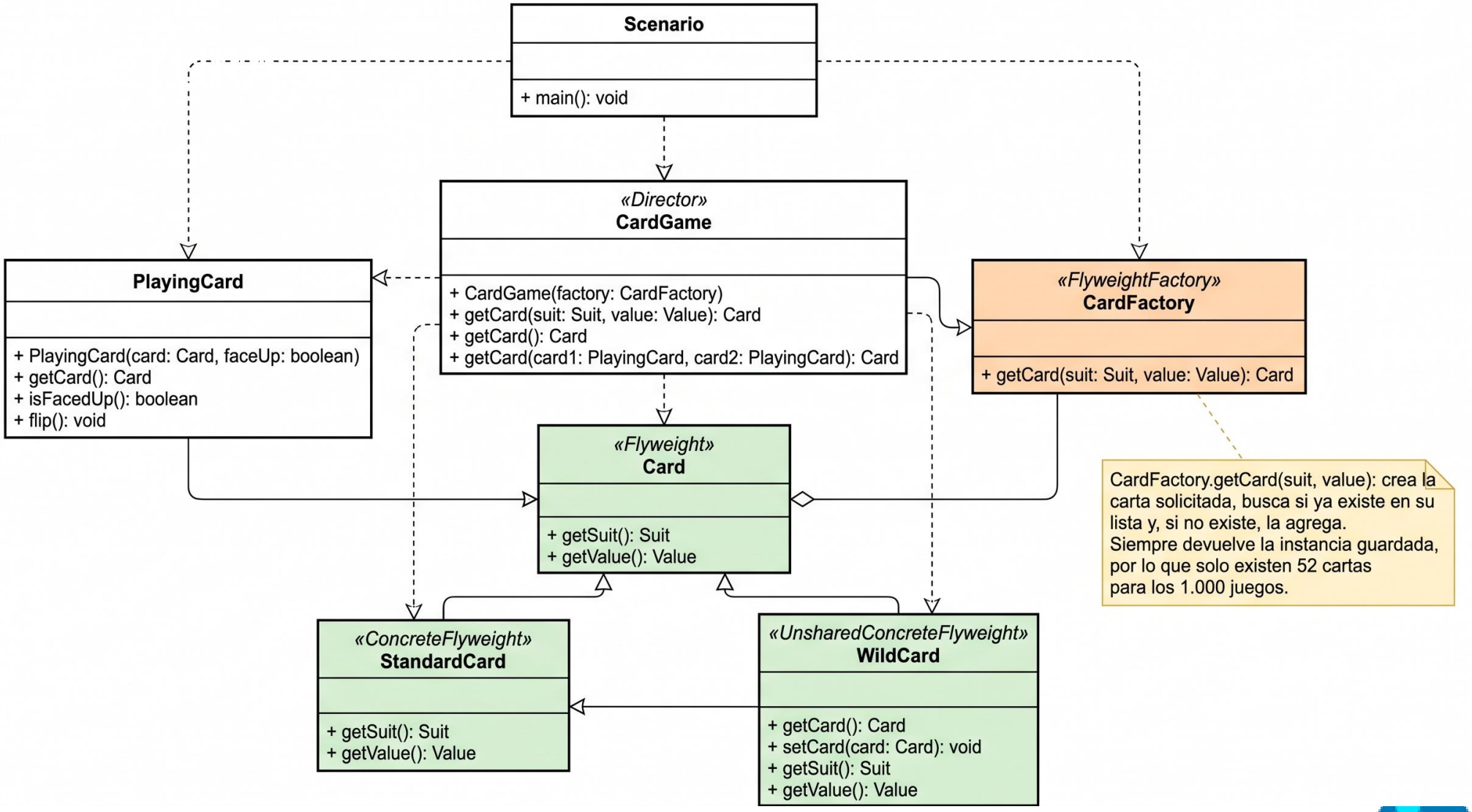
Ejercicio problema

En un torneo de cartas se disputan mil juegos, y cada juego es responsable de crear las 52 cartas de la baraja que necesita. Por eso hay que almacenar una gran cantidad de objetos iguales: 52.000 cartas mil ases de picas, mil reyes de corazones, etc.



Solución

UNIVERSIDAD Popular del cesar



¡Gracias!

UNIVERSIDAD
Popular del cesar